Rain BitTorrent 라이브러리 사용법 가이드

개요

Rain BitTorrent 클라이언트는 Go 언어로 구현된 고성능 BitTorrent 라이브러리로, 서버 환경에서 대량의 토렌트를 효율적으로 관리할 수 있도록 설계되었습니다. 이 문서는 Rain 라이브러리를 사용하여 BitTorrent 클라이언트를 구현하는 방법을 상세히 설명합니다.

기본 사용법

1. Session 생성

Session은 Rain BitTorrent 클라이언트의 핵심 컴포넌트로, 모든 토렌트와 리소스를 관리합니다.

```go
package main


import (
"log"
"github.com/cenkalti/rain/v2/torrent"
)


func main() {
// 기본 설정으로 Session 생성
cfg := torrent.DefaultConfig
// 필요에 따라 설정 수정
```

```go
cfg.DataDir = "./downloads"

cfg.Database = "./rain.db"

cfg.PortBegin = 20000

cfg.PortEnd = 30000

// Session 생성

session, err := torrent.NewSession(cfg)

if err != nil {

log.Fatal("Session 생성 실패:", err)

}

defer session.Close()

log.Println("Rain BitTorrent Session이 시작되었습니다.")

}
```

2. 토렌트 추가

2.1 토렌트 파일로 추가

```go
import (

"os"

"github.com/cenkalti/rain/v2/torrent"

)


func addTorrentFromFile(session *torrent.Session, filename string) (*torrent.T

// 토렌트 파일 열기
```

```go
file, err := os.Open(filename)
if err != nil {
return nil, err
}
defer file.Close()
// 토렌트 추가 옵션 설정
options := &torrent.AddTorrentOptions{
ID: "my-torrent-001", // 고유 ID (선택사항)
Stopped: false, // 자동 시작 여부
StopAfterDownload: false, // 다운로드 완료 후 중지 여부
StopAfterMetadata: false, // 메타데이터 다운로드 후 중지 여부
}
// 토렌트 추가
t, err := session.AddTorrent(file, options)
if err != nil {
return nil, err
}
log.Printf("토렌트 추가됨: %s (ID: %s)", t.Name(), t.ID())
return t, nil
}
```

2.2 마그넷 링크로 추가

```go
func addTorrentFromMagnet(session *torrent.Session, magnetURI string) (*torren
options := &torrent.AddTorrentOptions{
Stopped: false,
}
// 마그넷 링크로 토렌트 추가
t, err := session.AddURI(magnetURI, options)
if err != nil {
return nil, err
}
log.Printf("마그넷 토렌트 추가됨: %s", magnetURI)
return t, nil
}
```

2.3 HTTP URL로 추가

```go
func addTorrentFromURL(session *torrent.Session, url string) (*torrent.Torrent
options := &torrent.AddTorrentOptions{
Stopped: false,
}
// HTTP URL로 토렌트 추가
t, err := session.AddURI(url, options)
if err != nil {
return nil, err
```

```
}
log.Printf("URL 토렌트 추가됨: %s", url)
return t, nil
}
```

3. 토렌트 관리

3.1 토렌트 목록 조회

```
func listTorrents(session *torrent.Session) {
torrents := session.ListTorrents()
log.Printf("총 %d개의 토렌트가 있습니다:", len(torrents))
for i, t := range torrents {
stats := t.Stats()
log.Printf("%d. %s (상태: %s, 진행률: %.1f%%)",
i+1, t.Name(), stats.Status,
float64(stats.Pieces.Have*100)/float64(stats.Pieces.Total))
}
}
```

3.2 특정 토렌트 조회

```
func getTorrent(session *torrent.Session, id string) *torrent.Torrent {
t := session.GetTorrent(id)
```

```go
if t == nil {
log.Printf("ID '%s'인 토렌트를 찾을 수 없습니다.", id)
return nil
}
log.Printf("토렌트 발견: %s", t.Name())
return t
}
```

3.3 토렌트 제어

```go
func controlTorrent(t *torrent.Torrent) error {
// 토렌트 시작
err := t.Start()
if err != nil {
return fmt.Errorf("토렌트 시작 실패: %w", err)
}
// 잠시 대기
time.Sleep(5 * time.Second)
// 토렌트 중지
err = t.Stop()
if err != nil {
return fmt.Errorf("토렌트 중지 실패: %w", err)
}
return nil
```

```
}
```

3.4 토렌트 제거

```
func removeTorrent(session *torrent.Session, id string, keepData bool) error
err := session.RemoveTorrent(id, keepData)
if err != nil {
return fmt.Errorf("토렌트 제거 실패: %w", err)
}
if keepData {
log.Printf("토렌트 제거됨 (데이터 보존): %s", id)
} else {
log.Printf("토렌트 제거됨 (데이터 삭제): %s", id)
}
return nil
}
```

이벤트 처리 및 모니터링

1. 토렌트 상태 모니터링

```
func monitorTorrent(t *torrent.Torrent) {
ticker := time.NewTicker(5 * time.Second)
```

```go
defer ticker.Stop()
for {
select {
case <-ticker.C:
stats := t.Stats()
// 진행률 계산
progress := float64(0)
if stats.Pieces.Total > 0 {
progress = float64(stats.Pieces.Have*100) / float64(stats.Pieces.Total)
}
// ETA 계산
eta := "알 수 없음"
if stats.ETA != nil {
eta = stats.ETA.String()
}
log.Printf("[%s] 상태: %s, 진행률: %.1f%%, 속도: %d KB/s, ETA: %s, 피어: %d",
t.Name(),
stats.Status,
progress,
stats.Speed.Download/1024,
eta,
stats.Peers.Total)
case err := <-t.NotifyStop():
```

```go
if err != nil {
log.Printf("토렌트 중지됨 (오류): %v", err)
} else {
log.Printf("토렌트 정상 중지됨")
}
return
case <-t.NotifyComplete():
log.Printf("토렌트 다운로드 완료: %s", t.Name())
case <-t.NotifyMetadata():
log.Printf("메타데이터 다운로드 완료: %s", t.Name())
}
}
}
```

2. 세션 통계 모니터링

```go
func monitorSession(session *torrent.Session) {
ticker := time.NewTicker(10 * time.Second)
defer ticker.Stop()
for {
select {
case <-ticker.C:
stats := session.Stats()
```

```
log.Printf("세션 통계 - 토렌트: %d개, 피어: %d개, 업타임: %s",
len(session.ListTorrents()),
stats.Peers,
time.Duration(stats.Uptime)*time.Second)
log.Printf("속도 - 다운로드: %d KB/s, 업로드: %d KB/s",
stats.SpeedDownload/1024,
stats.SpeedUpload/1024)
log.Printf("캐시 - 읽기: %d MB, 쓰기: %d MB",
stats.ReadCacheSize/(1<<20),
stats.WriteCacheSize/(1<<20))
}
}
}
```

3. 완료 이벤트 처리

```
func handleTorrentCompletion(t *torrent.Torrent) {
go func() {
<-t.NotifyComplete()
log.Printf("토렌트 완료: %s", t.Name())
// 완료 후 처리 로직
stats := t.Stats()
log.Printf("최종 통계 - 크기: %d MB, 업로드: %d MB, 비율: %.2f",
```

```
stats.Bytes.Total/(1<<20),
stats.Bytes.Uploaded/(1<<20),
float64(stats.Bytes.Uploaded)/float64(stats.Bytes.Downloaded))
// 시딩 계속 또는 중지 결정
if shouldContinueSeeding(stats) {
log.Printf("시딩 계속: %s", t.Name())
} else {
log.Printf("토렌트 중지: %s", t.Name())
t.Stop()
}
}()
}

func shouldContinueSeeding(stats torrent.Stats) bool {
// 업로드 비율이 1.0 미만이면 계속 시딩
ratio := float64(stats.Bytes.Uploaded) / float64(stats.Bytes.Downloaded)
return ratio < 1.0
}
```

설정 옵션 상세 설명

1. 기본 설정

```go
func  createCustomConfig() torrent.Config {
cfg := torrent.DefaultConfig
// === 기본 디렉토리 설정 ===
cfg.Database = "~/rain/session.db"  // 세션 데이터베이스 파일
cfg.DataDir = "~/rain/data"  // 다운로드 디렉토리
cfg.DataDirIncludesTorrentID = true // 토렌트 ID별 하위 디렉토리 생성
// === 네트워크 설정 ===
cfg.Host = "0.0.0.0"  // 바인딩 호스트
cfg.PortBegin = 20000  // 포트 범위 시작
cfg.PortEnd = 30000  // 포트 범위 끝
cfg.MaxOpenFiles = 10240  // 최대 파일 핸들 수
// === 프로토콜 설정 ===
cfg.PEXEnabled = true // PEX 프로토콜 활성화
cfg.DHTEnabled = true // DHT 활성화
cfg.DHTPort = 7246  // DHT 포트
return  cfg
}
```

2. 성능 튜닝 설정

```go
func  createPerformanceConfig() torrent.Config {
cfg := torrent.DefaultConfig
// === 피어 연결 설정 ===
```

```
cfg.MaxPeerDial = 80  // 최대 아웃고잉 연결 수

cfg.MaxPeerAccept = 20  // 최대 인커밍 연결 수

cfg.UnchokedPeers = 3  // 언초크 피어 수

cfg.OptimisticUnchokedPeers = 1  // 낙관적 언초크 피어 수

// === I/O 성능 설정 ===

cfg.ParallelReads = 1  // 병렬 읽기 수

cfg.ParallelWrites = 1  // 병렬 쓰기 수

cfg.ReadCacheSize = 256 << 20  // 읽기 캐시 크기 (256MB)

cfg.WriteCacheSize = 1 << 30  // 쓰기 캐시 크기 (1GB)

cfg.ReadCacheTTL = 1 * time.Minute  // 읽기 캐시 TTL

// === 속도 제한 설정 ===

cfg.SpeedLimitDownload = 0  // 다운로드 속도 제한 (0=무제한)

cfg.SpeedLimitUpload = 0  // 업로드 속도 제한 (0=무제한)

return  cfg

}
```

3. 보안 설정

```
func  createSecureConfig() torrent.Config {

cfg := torrent.DefaultConfig

// === 암호화 설정 ===

cfg.DisableOutgoingEncryption = false // 아웃고잉 암호화 비활성화 여부

cfg.ForceOutgoingEncryption = false // 아웃고잉 암호화 강제 여부
```

```
cfg.ForceIncomingEncryption = false // 인커밍 암호화 강제 여부

// === 블랙리스트 설정 ===

cfg.BlocklistURL = "https://example.com/blocklist.txt"

cfg.BlocklistUpdateInterval = 24 * time.Hour

cfg.BlocklistEnabledForTrackers = true

cfg.BlocklistEnabledForOutgoingConnections = true

cfg.BlocklistEnabledForIncomingConnections = true

// === 프라이빗 토렌트 설정 ===

cfg.PrivatePeerIDPrefix = "-RN" + torrent.Version + "-"

cfg.PrivateExtensionHandshakeClientVersion = "Rain " + torrent.Version

cfg.TrackerHTTPPrivateUserAgent = "Rain/" + torrent.Version

return  cfg

}
```

4. RPC 서버 설정

```
func  createRPCConfig() torrent.Config {

cfg := torrent.DefaultConfig

// === RPC 서버 설정 ===

cfg.RPCEnabled = true // RPC 서버 활성화

cfg.RPCHost = "127.0.0.1"  // RPC 바인딩 호스트

cfg.RPCPort = 7246  // RPC 포트

cfg.RPCShutdownTimeout = 5 * time.Second  // RPC 서버 종료 타임아웃
```

```
return cfg
}
```

완전한 예제 프로그램

```
package main

import (
"context"
"log"
"os"
"os/signal"
"syscall"
"time"
"github.com/cenkalti/rain/v2/torrent"
)

func main() {
// 설정 생성
cfg := torrent.DefaultConfig
cfg.DataDir = "./downloads"
cfg.Database = "./rain.db"
```

```go
// Session 생성
session, err := torrent.NewSession(cfg)
if err != nil {
log.Fatal("Session 생성 실패:", err)
}
defer session.Close()
// 토렌트 추가 (예시)
if len(os.Args) > 1 {
torrentPath := os.Args[1]
t, err := addTorrentFromFile(session, torrentPath)
if err != nil {
log.Fatal("토렌트 추가 실패:", err)
}
// 모니터링 시작
go monitorTorrent(t)
go handleTorrentCompletion(t)
}
// 세션 모니터링 시작
go monitorSession(session)
// 종료 신호 대기
sigChan := make(chan os.Signal, 1)
signal.Notify(sigChan, syscall.SIGINT, syscall.SIGTERM)
log.Println("Rain BitTorrent 클라이언트가 시작되었습니다. Ctrl+C로 종료하세요.")
```

```
<-sigChan

log.Println("종료 신호를 받았습니다. 정리 중...")

// 모든 토렌트 중지

err = session.StopAll()

if  err != nil {

log.Printf("토렌트 중지 오류: %v", err)

}

log.Println("Rain BitTorrent 클라이언트가 종료되었습니다.")

}



// 앞서 정의한 함수들을 여기에 포함...
```

관련 문서

선행 지식

- 01-system-architecture.md - Rain의 전체 구조와 Session/Torrent 개념
- 03-01-torrent-lifecycle-process.md - 토렌트 생명주기 이해

관련 주제

- 07-02-cli-tool-implementation-analysis.md - CLI 도구 사용법과 비교
- 06-01-performance-optimization-analysis.md - 성능 튜닝 설정

후속 학습

- 04-01-networking-packages-analysis.md - 네트워크 구현 세부사항

- 05-01-dht-implementation-analysis.md - DHT 설정 세부사항
- 06-03-error-handling-recovery-analysis.md - 에러 처리 구현

> **네비게이션:** 이전: 에러 처리 및 복구 분석 | 목차 | 다음: CLI 도구 구현 분석